L'ordre des écritures

Plan d'un système qui garantit que les écritures manuelles et automatiques du module sont passées dans l'ordre, ligne à ligne, et une seule fois. Le principe tient en une phrase : on n'enregistre que la liste des écritures faites, et tout le reste — le curseur, l'état de chaque bloc, le Bilan, le Compte de Résultat, le grand journal — s'en déduit. La navigation reste libre : c'est l'ordre d'enregistrement qui est strict, jamais celui de la lecture. Proposition à valider, rien n'est encore écrit dans l'application.

> **CHANGEMENTS DEPUIS LA V1**
>
> **L'ordre est strict à l'intérieur des blocs aussi**, et non plus seulement entre eux : dans la plupart des blocs, l'ordre des lignes est pédagogique. Cela revient sur la liberté de saisie dans le bloc, décidée le 18 septembre. Conséquences répercutées ici : le curseur devient une ligne et non plus un bloc (sections 2 et 3), une seule écriture s'inscrit à la fois et le badge « À saisir » cesse d'être un guide pour devenir la cible (sections 7 et 8), la validation ne cherche plus parmi les lignes du bloc mais compare à la ligne attendue (section 5), et l'ordre des lignes dans `ecrsCompta` devient une donnée normative, à relire et à contrôler (section 14).
>
> **Le geste est tranché, lui aussi** : toutes les lignes restent cliquables, partout. Ce qui varie, c'est le repère et le droit d'écrire. Le cadre et le badge « À saisir » n'apparaissent que dans le chapitre où l'écriture a lieu, et l'écriture ne s'y fait que là : ailleurs, le popover s'ouvre quand même et porte le message et le lien vers le bon chapitre. Sections 1 et 10.
>
> Bonne nouvelle : les données actuelles s'y prêtent déjà. Les **vingt-neuf blocs, cent sept lignes** ont été passés au crible pour ce plan — identifiants tous uniques, un seul type par bloc, vingt-huit blocs équilibrés au centime (le vingt-neuvième est la ligne 120 du Chapitre 11, sans grand journal), et **tous les blocs manuels suivent déjà l'ordre débits puis crédits**. Les six blocs où un crédit précède un débit sont tous automatiques, et y regroupent plusieurs opérations. Rien à réécrire.

1. Ce que le système doit garantir

Trois garanties, et elles suffisent à tenir les chiffres du module :

1. **Aucune écriture ne peut être passée avant celle qui la précède.** Une tentative hors ordre est refusée, et le refus dit où reprendre.
2. **Aucune écriture ne peut être passée deux fois.** Un double clic, un retour en arrière, un rechargement de page ne rejouent rien.
3. **Les documents ne peuvent pas dériver.** Le Bilan, le Compte de Résultat et le grand journal sont recalculés depuis la liste des écritures faites ; il n'existe aucun total stocké qui pourrait ne plus correspondre.

Et ce que le système ne fait pas, volontairement : il n'enferme pas la lecture. L'apprenant ouvre le Chapitre 9 quand il veut, même s'il en est resté au Chapitre 5. Il y verra les panneaux du Chapitre 9 en lecture, inertes, avec un rappel de l'endroit où reprendre.

> LA RÈGLE EXACTE
>
> **Ordre strict de bout en bout : une seule écriture est attendue à la fois, dans tout le module.** Le bloc reste l'unité d'affichage, de légende et d'engagement automatique, mais il ne desserre plus rien : la deuxième ligne d'un bloc n'est saisissable qu'une fois la première passée.
>
> C'est un retour sur la décision du 18 septembre — l'ordre libre dans le bloc, la grille comme des mots croisés. Le motif est bon : dans la plupart des blocs, l'ordre des lignes porte un raisonnement, et le laisser prendre à l'envers revient à laisser lire une démonstration par la fin.
>
> **Ce que l'ordre strict coûte est payé par la main, pas par la porte.** Rien ne se ferme : l'apprenant clique sur la ligne qu'il veut, dans le chapitre qu'il veut, et le popover s'ouvre toujours — c'est déjà le lieu où il lit ce qui a été inscrit sur un compte. Deux choses seulement dépendent de l'endroit où il se trouve :
>
> - le **repère** — le cadre slate et le badge « À saisir » sur la ligne attendue — ne s'affiche que dans le chapitre où cette écriture a lieu ;
> - l'**écriture** ne se fait que là. Ailleurs, le popover explique en une phrase où elle se passe, et le lien y mène.
>
> C'est la bonne façon de tenir une contrainte : ce qui est imposé se voit au moment où il s'applique, et le refus n'est jamais une porte close mais un chemin indiqué.

---

2. Le principe : un curseur déduit, jamais stocké

La tentation serait d'enregistrer un numéro d'avancement — « l'apprenant en est au bloc 23 ». C'est le genre de compteur qui se désynchronise dès qu'on insère un bloc dans un chapitre, et qui oblige à une migration à chaque retouche du module. On fait l'inverse :

- on enregistre **la liste des identifiants d'écritures faites**, avec l'heure de chacune ;
- le **curseur** est la première ligne de la séquence absente de cette liste : une seule écriture est attendue à un instant donné, et le système sait toujours laquelle ;
- l'**état de chaque bloc** (fait, courant, à venir) se lit de la même façon, en regardant où tombe le curseur.

Deux conséquences heureuses. D'abord, rien de structurel n'est persisté : ni le numéro de chapitre, ni celui du bloc. Insérer un chapitre, déplacer un panneau, renuméroter n'invalide aucun avancement, parce que les identifiants (`tva-solde1-4455c`) ne bougent pas. Ensuite, le système est idempotent par construction : marquer une écriture déjà faite ne fait rien.

---

3. La séquence canonique

`ecrsCompta` est déjà la séquence du module : chapitres, puis blocs, puis lignes. On l'aplatit une fois, au démarrage, en **une liste de lignes** — c'est elle qui commande désormais — et en une liste de blocs qui pointe dedans, pour l'affichage.

```
// app/utils/sequenceEcrs.ts

export interface LigneEcrs extends EcritureCompta {
  rang: number          // position de la ligne dans le module — 0 à 106
  ch: number            // index de chapitre dans ecrsCompta
  bloc: number          // index de bloc dans le chapitre
  cleBloc: string       // le bloc auquel elle appartient
  surface: 'grille' | 'journal'   // où elle se saisit (gdJalTodo)
}

export interface BlocEcrs {
  cle: string           // l'id de sa première ligne — stable, jamais un index
  ch: number
  bloc: number
  type: 'todo' | 'auto'
  debut: number         // rang de sa première ligne
  lignes: LigneEcrs[]
}

export function sequenceLignes(ecrs: EcritureCompta[][][]): LigneEcrs[]
export function blocsDe(lignes: LigneEcrs[]): BlocEcrs[]
export function empreinte(lignes: LigneEcrs[]): string   // ids concaténés, hashés
```

Le `rang` est un numéro de travail, calculé à chaque démarrage : il n'est jamais écrit dans Firebase, jamais comparé d'une session à l'autre. La **clé d'un bloc est l'identifiant de sa première ligne**, pas `« 5.3 »` — un index se décale, un identifiant non. L'**empreinte** est une petite somme de contrôle de toute la séquence ; elle sert au chargement, pour savoir en une comparaison si le module a changé depuis la dernière session.

`surface` mérite un mot : trois lignes manuelles du Chapitre 5 et une du Chapitre 8 portent `gdJalTodo` et n'ont pas de case dans la grille — elles se saisissent au grand journal. Avec un ordre strict, le système sait donc à tout instant non seulement quelle ligne est attendue, mais **où** elle s'écrit, et peut le dire plutôt que de laisser chercher.

---

4. Ce qui est enregistré dans Firebase

Un document par apprenant et par module, lu une fois, écrit à chaque bloc. Pas de sous-collection d'écritures : une centaine de documents à lire pour un module, ce serait une centaine de lectures facturées pour cinq kilo-octets d'information.

```
apprenants/{uid}/avancement/{moduleId}

{
  empreinte: 'a3f19c',                  // celle de la séquence au moment de l'écriture
  faites: [                             // dans l'ordre d'inscription
    { id: 'nais-cap-512d', ts: 1789884044716 },
    { id: 'nais-cap-101c', ts: 1789884044720 },
    …
  ],
  majLe: serverTimestamp()
}
```

Le module complet fait 107 écritures ; le document plafonne à quelques kilo-octets, très loin du mégaoctet permis par Firestore. Les règles de sécurité tiennent en une ligne : `allow read, write: if request.auth.uid == uid`.

L'horodatage de chaque ligne n'est pas décoratif : c'est lui qui donne l'ordre du grand livre dans le popover de la grille, et le `timestamp` attendu par `BilanCompteDeResultat.vue`. Les dates comptables, elles, restent dans `gdJal.libelle` — ce sont deux choses différentes, et il faut qu'elles le restent.

> VALIDATION AU CHARGEMENT, PLUTÔT QU'UNE MIGRATION
>
> L'ordre strict rend la validation triviale : après lecture, on confronte `faites` à la séquence, ligne par ligne, et **on tronque au premier désaccord**. L'invariant — les écritures faites forment un début de séquence — est donc rétabli à chaque ouverture, sans code de migration à écrire. Si la liste a été tronquée, on réécrit le document nettoyé. Au pire, l'apprenant refait les quelques lignes du bloc en cours.

---

5. Le composable

Le moteur d'ordre ne connaît ni Firebase, ni Vue, ni le module 1 : il reçoit la séquence des écritures et un service de persistance. C'est ce qui le rend testable et réutilisable pour les modules suivants.

```
// app/composables/useAvancementEcrs.ts

export interface ServiceAvancement {
  lire(moduleId: string): Promise<Avancement | null>
  ecrire(moduleId: string, a: Avancement): Promise<void>
  effacer(moduleId: string): Promise<void>
}

export type Refus =
  | { motif: 'pas-prete' }                       // avancement pas encore lu
  | { motif: 'inconnue' }                        // id absent de la séquence
  | { motif: 'hors-ordre', attendue: LigneEcrs }  // une écriture précède celle-ci

export type Resultat =
  | { ok: true, deja?: true }                    // idempotent
  | ({ ok: false } & Refus)

export function creerAvancement(opts: {
  moduleId: string
  lignes: LigneEcrs[]
  service: ServiceAvancement
}) {
  return {
    // état
    pret, enChargement, erreur,
    faites,              // Map<id, ts> en lecture seule
    courante,            // LigneEcrs | null (null = module terminé)
    blocCourant,         // le bloc de `courante`
    termine,

    // lecture
    estFaite,            // (id) => boolean
    estAttendue,         // (id) => boolean  — la seule ligne saisissable
    etatBloc,            // (cle) => 'fait' | 'courant' | 'a-venir'
    avancementBloc,      // (cle) => { total, faites }  — pour le compteur

    // actions
    charger,             // idempotent, appelé au premier accès
    marquer,             // (id) => Resultat   — refuse tout sauf `courante`
    marquerBloc,         // (cle) => Resultat  — un bloc automatique entier
    reinitialiser,
  }
}
```

Un seul exemplaire. Un composable appelé dans quinze composants crée quinze états s'il n'est pas protégé. Plutôt qu'une variable globale dans le module TypeScript, on laisse Pinia faire son travail : `creerAvancement` est appelé une fois, dans `store/compta.ts`, et les composants n'utilisent que le store. Le composable reste une fabrique sans état global, donc testable en boucle.

`marquer` n'accepte qu'un identifiant : celui de `courante`. Plus de recherche parmi les lignes non faites du bloc, plus de candidats : l'écriture attendue est connue, la comparaison est une égalité. C'est le bénéfice discret de votre arbitrage — la validation d'une saisie se réduit à deux questions, est-ce la bonne ligne, est-ce le bon montant, et le second contrôle reste celui du 18 septembre : un montant faux ne s'inscrit pas.

Les refus sont des données, pas des chaînes. Le texte est fabriqué à part, par un `messageRefus(refus)`, pour pouvoir le relire comme on relit une phrase du module. Le refus `hors-ordre` porte la ligne attendue, donc le message peut la nommer — « Cette écriture vient plus tard. Commencez par 512 Banque. » plutôt qu'un refus qui laisse chercher.

---

6. Les documents se déduisent

`bilan`, `cr`, `gdjal`, `resultatAvantImpot` et `resultatNet` deviennent des `computed` du store, construits en parcourant `faites`. Leur forme ne change pas — `{ cpt, somme, timestamp }` — donc `BilanCompteDeResultat.vue` n'a rien à apprendre.

La conversion reste celle du 20 septembre : `somme` est l'effet signé sur la case, jamais un débit ou un crédit. Actif et charges montent au débit, passif et produits au crédit ; les lignes 4456 et 4457 n'ont pas de case et ne vont qu'au journal.

> **UN PLAN DE COMPTES, UN SEUL**
>
> Cette conversion demande de savoir, pour chaque numéro de compte, son document et son côté. Aujourd'hui l'information est dans la chaîne `doc` de chaque écriture (« Bilan — 512 Banque »), c'est-à-dire répétée 107 fois et destinée à l'affichage. Il faut une table unique, `app/utils/planComptes.ts`, équivalente au `A` du banc : numéro, document, côté, nom, précision, ordre d'affichage. Le `doc` des écritures peut rester pour le texte des panneaux, à condition qu'un contrôle vérifie qu'il concorde avec le plan — sinon on aura tôt ou tard un compte au bon endroit dans la grille et au mauvais dans le panneau.

---

7. Les trois états d'un bloc

Tout l'affichage se déduit de `etatBloc(cle)`. Aucun panneau ne décide seul de son état, et aucun ne connaît les autres.

ÉTAT	PANNEAU MANUEL	PANNEAU AUTOMATIQUE	GRILLE
Fait	Lignes cochées, ton discret, pas de compteur	Lignes visibles et cochées, pas de bouton	Les lignes de ce bloc sont dans les totaux
Courant	Ligne attendue mise en avant, checks au fur et à mesure, compteur « 1 / 3 »	Bouton d'enregistrement, puis défilé ligne à ligne	Repère sur la ligne attendue, et elle seule s'écrit
À venir	Lignes en lecture, inertes, rappel de reprise	Bouton désactivé, rappel de reprise	Aucun repère ; le popover s'ouvre, et renvoie au bon chapitre

Deux précisions. La colonne « grille » suppose que la page ouverte est celle du bloc : le repère et l'écriture suivent le chapitre, pas seulement le curseur — une page de chapitre antérieur, dont tous les blocs sont faits, n'affiche donc

aucun repère non plus. Et le rappel de reprise n'apparaît qu'une fois par page, pas une fois par panneau : une page qui porte six blocs à venir n'a pas six bandeaux à afficher.

---

8. Le panneau manuel

`PanneauEcrsTodo.vue` garde sa structure et son aspect. Ce qui s'ajoute :

- Un **check par ligne** devant le libellé. Coché : `<UIcon name="mdi:check-circle-outline" />` ; en attente, un point médian de même gabarit, pour que la hauteur de ligne ne bouge pas quand le check apparaît. La couleur du coché suit le panneau (sky), pas un vert nouveau.
- Un **compteur** sous l'en-tête, « 1 / 3 lignes passées », en `aria-live="polite"` : c'est ce qu'entend un lecteur d'écran quand une ligne est acceptée dans la grille.
- La **ligne attendue** se distingue des autres lignes du bloc — c'est la contrepartie de l'ordre strict : ce qui est imposé doit être montré. Le même repère que dans la grille, pour qu'on relie les deux d'un coup d'œil, sans reprendre le cadre slate qui appartient à la case. À trancher avec la mise en avant du bloc courant, ci-dessous : une seule chose doit attirer l'œil sur la page, et le badge de la grille est déjà là.
- Le **bloc courant** reçoit un liseré ou un fond très léger — l'apprenant qui revient sur une page à quatre blocs doit voir en un coup d'œil lequel l'attend. À trancher : rien de plus qu'un liseré, pour ne pas ajouter une couleur au vocabulaire.
- Le panneau **à venir** passe en opacité réduite et perd le pointeur ; ses lignes restent lisibles, puisque le texte du chapitre en parle.

Pas de bouton dans ce panneau : la saisie se fait dans la grille, comme décidé le 18. Le panneau est le lieu de la consigne et du décompte, la grille celui du geste.

---

9. Le panneau automatique

`PanneauEcrsAuto.vue` reçoit un bouton — un `UButton` neutral, comme celui de `BilanCompteDeResultat.vue` — et le défilé du banc : les lignes s'inscrivent une à une, environ 750 ms, avec « Tout afficher » pour couper court, et un compteur qui remplace le « Rien ne s'inscrit avant que vous actionniez le panneau ».

> **LE CLIC ENGAGE TOUT LE BLOC**
>
> `marquerBloc` enregistre **l'ensemble des lignes d'un coup**. Le défilé n'est qu'une présentation, pilotée par un compteur local au composant. Autrement dit, si l'apprenant change de page au milieu de l'animation, ou recharge, le bloc est passé et la grille est juste ; il n'existe jamais de bloc automatique à moitié enregistré. C'est la seule façon simple de tenir la garantie n° 3, et ça évite une écriture Firebase par ligne.

Quand un panneau automatique porte plusieurs blocs (`:index-bloc-ecrs="[1]"` accepte déjà un tableau), le bouton les enchaîne dans l'ordre. Le contrôle n° 5 vérifie que ces blocs sont bien consécutifs dans la séquence : un panneau qui regrouperait les blocs 1 et 3 en laissant le 2 ailleurs sur la page ferait mentir le curseur.

---

10. La grille et le geste de saisie

Le branchement est court, et l'ordre strict le raccourcit encore. La grille demande au store `courante` : une ligne, au format `gdJal`, exactement la `consigne` qu'elle attend déjà. C'est la seule case où un montant s'inscrit — les autres

s'ouvrent, mais pour lire. Quand la saisie est acceptée, elle appelle `marquer(id)` au lieu de pousser elle-même dans `bilanCr`. Le store recalcule, les totaux s'animent, le check apparaît dans le panneau, et le composant n'a plus d'état à posséder — ni `emit('termine')` à envoyer, puisque la fin d'un bloc se lit dans le curseur.

Il faut donc que la consigne porte l'`id` de la ligne, ce que votre interface `gdJal` prévoit déjà en optionnel. On le rend obligatoire ici.

Les quatre lignes sans case. Trois au Chapitre 5, une au Chapitre 8 : quand `courante.surface` vaut `'journal'`, la grille n'a aucune case à ouvrir et c'est le grand journal qui prend la main, avec le choix de la colonne — débit ou crédit — que le banc prévoyait déjà. La grille le dit alors en une ligne, sous les documents, plutôt que de laisser l'apprenant chercher une case qui n'existe pas. C'est un cas que l'ordre libre rendait confus et que l'ordre strict rend lisible : l'application sait précisément quand ce moment arrive.

La saisie suit la page, la lecture suit l'apprenant

La grille est dans le volet de simulation, présent sur toutes les pages. L'écriture, elle, se passe dans son chapitre : la consigne est dans le texte, et saisir 1 200 € de TVA en lisant les salaires n'apprendrait rien. La grille compare donc deux choses — le chapitre de la page ouverte et celui de `courante` — et n'en tire que trois cas.

```
// dans le store : une question, une réponse

etatSaisie(chPage)
  → 'ouverte'          // chPage === courante.ch — le repère s'affiche
  → 'autre-chapitre'   // l'écriture attendue se passe ailleurs
  → 'terminee'         // plus rien à passer dans le module
```

Le popover répond dans tous les cas, parce qu'il est déjà le lieu où l'on vient lire ce qu'une ligne porte. Il prend l'un de trois visages, et l'historique du compte reste affiché dessous dans les trois :

LA LIGNE CLIQUÉE	CE QUE PORTE LE POPOVER
C'est la ligne attendue, et nous sommes dans son chapitre	Le champ du montant, et rien d'autre — le geste du 18 septembre, inchangé
L'écriture attendue se passe dans un autre chapitre	Une phrase qui dit lequel, et un lien qui y mène
Nous sommes au bon endroit, mais ce n'est pas la ligne attendue	Une phrase qui nomme la ligne par laquelle commencer

Ce sont trois phrases à écrire, pas trois mécanismes : le store donne à chaque fois de quoi les construire — le chapitre visé et son lien, ou la ligne attendue. Elles vont dans `messageRefus.ts` avec les autres, et se relisent ensemble.

Une conséquence agréable : la grille n'a plus besoin de bandeau. Le rappel de reprise reste utile sur la page, au-dessus des panneaux d'un chapitre qu'on ne peut pas encore remplir, mais dans le volet de simulation, le message arrive au moment où l'apprenant le demande — quand il clique — plutôt que d'occuper la place en permanence.

11. Savoir où mène le lien

Deux endroits pointent vers le chapitre à reprendre : le popover de la grille, en une phrase, et le bandeau `RappelReprise.vue` au-dessus des panneaux d'un chapitre encore inaccessible. Dans les deux, le lien est un vrai `NuxtLink`, et la phrase nomme le chapitre par son titre plutôt que par son numéro.

Pour l'écrire, il faut relier un bloc à sa page. Inutile d'ajouter une table : les pages portent déjà `module_id`, `ordre` et `titre` dans `definePageMeta`. On construit la liste des chapitres depuis `router.getRoutes()`, filtrée sur le module et triée sur `ordre`.

> UN DÉCALAGE À VÉRIFIER
>
> `la-tva.vue` déclare `ordre: 6` et ses panneaux `:index-chapitre="5"` : l'avant-propos occupe une place dans l'ordre des pages et une place dans `ecrsCompta`, mais pas la même. L'invariant à poser et à contrôler est donc `indexChapitre === ordre - 1`. Mieux encore, à trancher : **que le panneau n'ait plus à déclarer son chapitre**. `ModuleChapter` le fournit par `provide` depuis la route, et chaque panneau ne déclare que son bloc. Cela retire d'un coup treize occasions de se tromper, au prix d'une passe sur les treize pages.

12. Le reset

Portée : un module, c'est-à-dire le document Firebase et rien d'autre. `reinitialiser()` efface le document, vide `faites`, et le curseur retombe sur le premier bloc du Chapitre 2. La position de lecture n'est pas touchée : on efface un avancement, on ne renvoie personne à la page 1.

- **Où** : dans le volet de simulation, sous un menu discret, et non dans le fil du texte. À trancher : une page de réglages du module serait plus juste si d'autres remises à zéro arrivent.
- **Confirmation** : jamais de modale. Un `UPopover` à deux temps — « Effacer tout l'avancement de ce module ? » puis un bouton d'accord —, puis un toast qui confirme.
- **En développement** : les mêmes fonctions exposées autrement, pour ne pas fabriquer un chemin de code parallèle. Un `allerAuBloc(cle)` qui rejoue tous les blocs précédents en une fois — l'équivalent du sélecteur « Aller à » du banc — est indispensable pour tester le Chapitre 11 sans repasser cent écritures. À garder derrière `import.meta.dev`.

13. Lecture, écriture, pannes

Lecture

Une fois, au premier appel du store après connexion, comme vous le prévoyez. Tant que `pret` est faux, **les panneaux et la grille sont inertes** — un `USkeleton` sur le compteur suffit. C'est le piège principal de ce genre de système : un clic sur un panneau automatique avant la fin de la lecture écraserait l'avancement par un avancement vide. `marquer` et `marquerBloc` refusent avec `pas-prete` ; ils ne doivent pas se contenter de compter sur l'interface.

Écriture

Optimiste, puis persistée : l'interface avance immédiatement, l'écriture part derrière. Un `setDoc` du document entier, pas un `arrayUnion` — l'invariant de préfixe rend le dernier écrivain légitime, et des objets `{ id, ts }` se prêtent mal aux opérations atomiques de tableau. Trois précautions :

- **Groupement** : un débounce d'environ 400 ms, pour qu'un bloc de trois lignes saisies rapidement ne fasse qu'une écriture.
- **File sérialisée** : une écriture à la fois, la suivante attend la précédente. Deux `setDoc` concurrents sur le même document, c'est l'unique manière de perdre une ligne.
- **Échec** : trois tentatives espacées, puis un message sobre — « L'avancement n'a pas pu être enregistré ; il le sera à la prochaine occasion » — et la file reste en mémoire. Rien ne se bloque : l'apprenant continue, ses documents sont justes à l'écran.

Deux onglets

Sans temps réel, deux onglets ouverts peuvent se marcher dessus. La conséquence est bénigne par construction : comme les blocs faits forment toujours un début de séquence, la validation au chargement remet tout d'aplomb, et la perte maximale est un bloc à refaire. Le dire ici plutôt que d'ajouter un verrou.

À trancher : un miroir dans `localStorage`, fusionné au chargement par union des identifiants, sauverait une session passée hors ligne. C'est vingt lignes et une source de vérité de plus. Mon avis : pas au module 1.

---

14. Les sept contrôles qui garantissent réellement l'ordre

Un curseur déduit ne vaut que ce que vaut la séquence dont il est déduit — et depuis que l'ordre des lignes commande, `ecrsCompta` n'est plus seulement une source d'affichage : c'est le scénario du module. Ces contrôles sont la vraie garantie ; ils tiennent dans un fichier de tests et doivent tourner à chaque modification du fichier.

Trois d'entre eux — les n° 1, n° 2 et n° 3 — ont été passés à la main sur `compta.ts` pour préparer ce plan, ainsi que le n° 7 : **tous quatre passent**. Les n° 4, n° 5 et n° 6 demandent respectivement le rejeu, la lecture des treize pages et le plan de comptes ; ils restent à faire.

N°	CONTRÔLE	CE QU'IL ATTRAPE
1	Tous les `id` du module sont uniques	Un copier-coller de bloc : deux lignes au même id, l'une coche l'autre
2	Chaque bloc n'a qu'un seul `type`	Un bloc mi-manuel mi-automatique, que rien n'afficherait correctement
3	Chaque bloc est équilibré au journal, débits = crédits	Une faute de frappe dans un montant. Seule exception admise : la ligne 120 du Chapitre 11, sans `gdJal`
4	Le rejeu de toute la séquence donne les chiffres du déroulé	Toute dérive : le Bilan doit boucler sur 130 410 € et le total de contrôle tomber sur 1 294 694,00 €
5	Chaque bloc est référencé par exactement un panneau, sur la page de son chapitre, dans l'ordre	Un bloc oublié — qui bloquerait le curseur à jamais — ou deux panneaux sur le même bloc
6	Tous les `cpt` existent dans le plan de comptes, et `doc` concorde avec lui	Un compte au bon endroit dans la grille et au mauvais dans le panneau
7	Dans un bloc manuel, les débits précèdent les crédits	Une ligne déplacée par inadvertance : depuis que l'ordre commande, un bloc réordonné change ce que l'apprenant doit faire, silencieusement

Le n° 5 se fait par lecture des fichiers `.vue` du module : on y cherche les `indexChapitre` et `indexBlocEcrs`. C'est du même ordre que `verif_v28.py` côté documents — un contrôle qui lit les sources et refuse de mentir. Le n° 4 est le plus utile des sept : c'est le pont entre l'application et le déroulé.

Le n° 7 est neuf, et c'est le prix de l'ordre strict. Il ne vaut que pour les blocs manuels : les six blocs automatiques où un crédit précède un débit groupent plusieurs opérations — une vente, puis sa taxe, puis un achat — et leur ordre est celui du récit, pas celui de l'écriture. Ce contrôle ne dit d'ailleurs pas que l'ordre est *pédagogique*, seulement qu'il n'a pas bougé sans qu'on le veuille. La question pédagogique, elle, se règle une fois, en relisant les vingt-deux blocs manuels — quarante-neuf lignes — comme on relit une consigne.

---

15. Les fichiers

```
app/
  utils/
    sequenceEcrs.ts        aplatissement, clés de bloc, empreinte
    planComptes.ts         cpt → document, côté, nom, précision, ordre
    messageRefus.ts        les refus en français, relus comme du texte
  composables/
    useAvancementEcrs.ts   le moteur d'ordre, sans Vue ni Firebase
  services/
    avancementFirestore.ts lire / écrire / effacer, file et débounce
  stores/
    compta.ts              ecrsCompta + creerAvancement() + bilan, cr, gdjal
  components/m1/
    PanneauEcrsTodo.vue    + checks, compteur, états
    PanneauEcrsAuto.vue    + bouton, défilé, états
    BilanCompteDeResultat.vue   consigne et marquer() viennent du store
    RappelReprise.vue      « Reprenez au Chapitre 5 », partagé
test/
  sequence.spec.ts         les six contrôles
  avancement.spec.ts       ordre, refus, idempotence, reset, service factice
```

---

16. Les huit étapes

Dans cet ordre : chacune est vérifiable seule, et les quatre premières ne touchent pas à l'interface.

1. **La séquence et les contrôles.** `sequenceEcrs.ts`, `planComptes.ts`, les six contrôles en tests. Livrable : la séquence est sûre, et on sait qu'elle boucle sur les chiffres du déroulé.
2. **Le composable**, avec un service factice en mémoire. Tests d'ordre, de refus, d'idempotence, de troncature au chargement.
3. **Les dérivés du store** — bilan, cr, gdjal, résultats — et le rejeu vérifié.
4. **Le service Firestore** : lecture unique, file, débounce, réessais, règles de sécurité.
5. **Le panneau manuel** : checks, compteur, trois états.
6. **Le panneau automatique** : bouton, engagement atomique, défilé, « Tout afficher ».
7. **La grille** : consigne prise du store, repère sur la ligne attendue, `marquer()` à la saisie juste, et les trois visages du popover.
8. **Reset, rappel de reprise, accessibilité** : `aria-live`, focus après une ligne acceptée, téléphone, mouvement réduit.

17. Ce qui reste à trancher

- [ ] Relecture des vingt-deux blocs manuels : leur ordre de lignes est désormais une consigne — section 14.
- [ ] Le clic d'un panneau automatique engage tout le bloc, l'animation n'est que présentation — section 9.
- [ ] Une seule table de comptes ; `doc` gardé pour l'affichage, contrôlé contre elle — section 6.
- [ ] Le chapitre fourni par la page plutôt que déclaré par chaque panneau — section 11.
- [ ] Emplacement et libellé du reset ; page de réglages ou menu du volet — section 12.
- [ ] **Les trois phrases du popover**, celle du rappel de reprise et celle du compteur, à écrire comme du texte de module — sections 10 et 11.
- [ ] Miroir `localStorage` : oui ou non au module 1 — section 13.
- [ ] Ce qui montre le bloc courant dans le texte : le repère de la grille suffit-il, ou faut-il distinguer le panneau aussi — section 8.

Aucune de ces huit questions n'empêche de commencer : les quatre premières étapes ne dépendent que de la section 3 et de la section 4, toutes deux indépendantes des arbitrages d'interface. La première, en revanche, est une relecture éditoriale, et elle ne demande pas d'attendre le code.